CNSCC.160 – Focus on Speech Processing

Practical Session for Week 5

LPC-based speech processing using MATLAB

Introduction This lab session aims to familiarize students with the principles of Linear Predictive Coding (LPC) and its application in speech processing using MATLAB. LPC is a widely used technique in speech analysis and synthesis, commonly applied in speech compression, speech recognition, and vocoders. This session involves conducting experiments to understand LPC analysis and synthesis, analyze residual signals, and examine synthesized speech using different excitation signals.

Theoretical Background Linear Predictive Coding (LPC) is a method for encoding speech signals by estimating the parameters of an all-pole filter that represents the vocal tract. It assumes that the speech signal is produced by filtering an excitation signal through a vocal tract filter. The model is commonly defined by a "p" order recursive (IIR) digital filter, where typically p=10, meaning the filter is defined using 10 coefficients.

Vocal Tract Modelling and LPC Analysis

The human vocal tract acts as a dynamic filter that shapes the sound produced by the glottal source. LPC models the vocal tract as a series of resonances (formants) using an autoregressive model. The filter coefficients derived from LPC analysis describe how the spectral characteristics of speech change over time.

LPC analysis operates on short overlapping frames of speech (typically 20-30 milliseconds) to account for the quasi-stationary nature of speech signals. The algorithm estimates the LPC coefficients by minimizing the squared difference between the actual speech sample and its predicted value based on past samples.

The computed LPC coefficients effectively capture the shape of the vocal tract and form the basis for speech compression, enhancement, and synthesis applications.

LPC Analysis Outputs: Understanding a, g, and e

When performing LPC analysis using the function `LPC_analysis(d, N)`, the following outputs are obtained:

- **a (LPC filter coefficients):** These coefficients represent the spectral envelope of the speech signal. They define the autoregressive filter that models the vocal tract.
- **g (Gain factor):** This parameter represents the overall energy or amplitude scaling factor of the LPC model. It ensures that the synthesized speech maintains the correct loudness relative to the original signal.

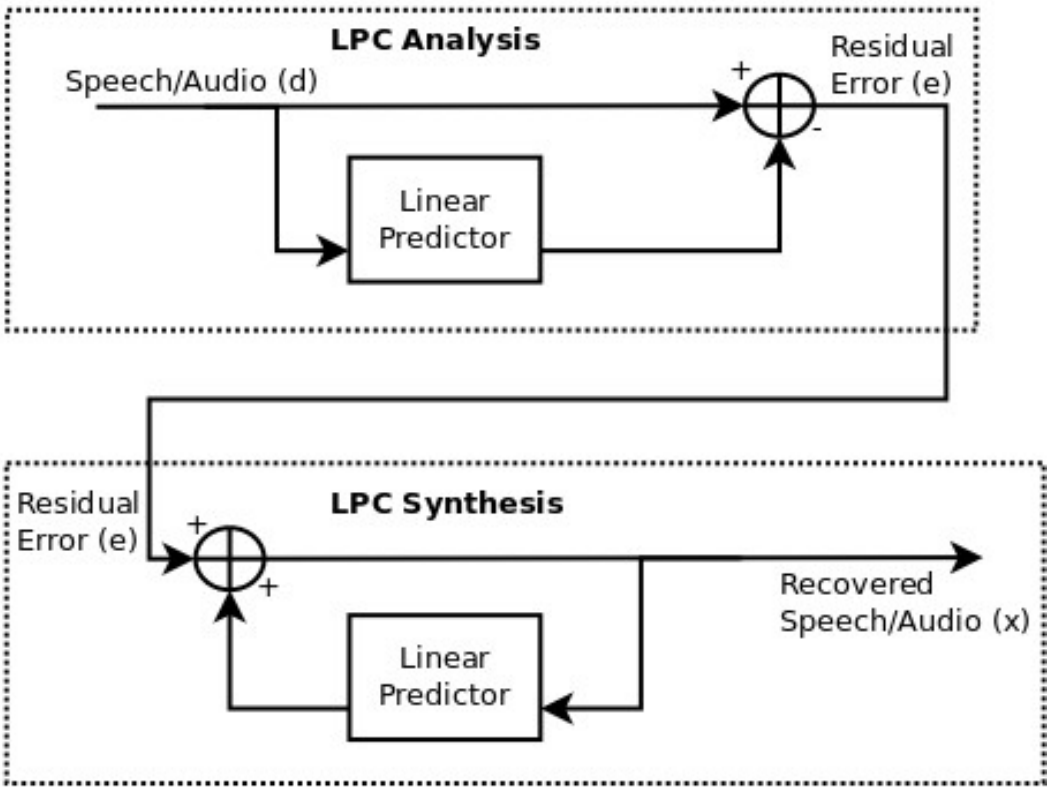
- **e (Residual error signal):** Also known as the excitation signal, this represents the difference between the predicted and actual speech samples. It contains crucial information about voiced and unvoiced characteristics and is necessary for high-quality speech reconstruction.

Excitation Signal and the Binary Excitation Model

The vocal tract is excited by a source signal, which determines whether the sound is voiced or unvoiced:

1. **Unvoiced Speech Sounds:** These are produced using turbulent airflow (e.g., /s/, /f/) and are best represented by white noise.
2. **Voiced Speech Sounds:** These are periodic sounds (e.g., vowels, /m/, /n/) that are driven by the quasi-periodic vibrations of the vocal cords.

This dual representation of excitation is referred to as the **Binary Excitation Model**, a key principle used in vocoder-based low-bit-rate speech coding systems. The excitation signal is crucial for determining speech quality when reconstructing signals from LPC parameters.



Residual Signal and LPC Synthesis

The **residual signal (e)** is the error produced after filtering out the spectral envelope from the speech signal using an inverse LPC filter. This residual contains the fine details of the excitation signal and, when reintroduced into an LPC synthesis filter, allows for near-perfect reconstruction of the original speech.

Objective

The purpose of this experiment is to:

1. **Verify** that LPC analysis and synthesis cancel each other out when used sequentially.
2. **Investigate** how different excitation signals, such as noise or music, affect the synthesis process.

Lab Instructions In this lab, we will use two MATLAB functions: `LPC_analysis.m` and `LPC_synthesis.m`. These functions facilitate LPC-based speech analysis and synthesis. The workflow follows these steps:

At the transmitter

```
function [a,g,e]=LPC_analysis(d,N);
```

operates on the input signal in MATLAB variable "d" to generate the LPC parameters. The input and output parameters are:

d	The input signal.
N	The input order of the LPC filter.
a	The output LPC filter coefficients.
g	The output gain of the filter.
e	The output residual error signal.

At the receiver

```
function x = LPC_synthesis(a,g,e);
```

is used to synthesize and therefore recover speech, where:

x	The output recovered speech signal.

Error! Reference source not found. Figure shows the LPC system process. LPC analysis yields the residual error "e". This error signal is directed to the input of the LPC synthesis process. Therefore the LPC synthesis process operates on "e" to produce the recovered signal "x".

Exercises

1. Clear all data currently stored on the workspace:

```
clear all;
```

2. Load one of the sample speech WAV files:

```
[d,sr]=wavread('speech01.wav');
```

   If you have newer version of matlab then use

```
[d,sr]=audioread('speech01.wav');
```

3. Listen to the original signal:

```
soundsc(d,sr);
```

4. Obtain outputs “a”, “g” and “e” for a 10th-order LPC model using the function “LPC_analysis”.

```
[a,g,e] = LPC_analysis(d,10);
```

5. Without using the residual “e”, generate a noise-excited LPC synthesized signal:

```
rd = LPC_synthesis(a,g);
```

   As we are not, in this case, explicitly supplying an input excitation signal, the LPC synthesis program internally creates one using a “noise” signal generated by a random number generator.

6. Listen to this **noise-excited LPC** synthesized signal and describe how this compares to the original speech in “d”:

```
soundsc(rd,sr);
```

7. Generate the **residual-excited LPC** synthesis speech signal (i.e. using the ‘e’ excitation signal as input to the LPC synthesis process)

```
rde = LPC_synthesis(a,g,e);
```

8. Listen to the “rde” recovered signal, which should be perceptually identical to original

```
soundsc(rde,sr);
```

(go to the next page for Part II)

Part II

9. Compare the spectrograms of these three types of signals, and comment on their differences.

```
subplot(3,1,1);
specgram(d,256,sr);
subplot(3,1,2);
specgram(rd,256,sr);
subplot(3,1,3);
specgram(e,256,sr);
```

10. Play the residual signal.

```
soundsc(e,sr);
```

11. Synthesize an artificial signal at the output of the previous LPC synthesis filter (using the "a" coefficients), with an **excitation input signal derived from the LPC analysis of a file containing music**. Comment on the intelligibility of the synthesized signal.

```
[d1,r1]=audioread('bach16.wav'); % MUSIC SIGNAL
d1 = resample(d1,1,2);
r1 = r1/2;
[a1,g1,e1] = LPC_analysis(d1,10);
% USE e1 FROM THE MUSIC SIGNAL TO REPRODUCE THE SPEECH SIGNAL
rde1 = LPC_synthesis(a,g,e1);
soundsc(rde1,r1);
```

12. Compare the original speech waveform with that of the residual error and LPC synthesis recovered signals. Thus plot "d", "e", "rde" as well as "e1" and "rde1".

```
subplot(5,1,1);
plot(d(2000+[1:1000]));
title('Original Signal (speech)')
xlabel('Time');
ylabel('Amplitude');
subplot(5,1,2);
plot(e(2000+[1:1000]));
title('Residual Error (speech)');
xlabel('Time');
ylabel('Amplitude');
subplot(5,1,3);
plot(rde(2000+[1:1000]));
title('Recovered Signal (speech)');
xlabel('Time');
ylabel('Amplitude');
subplot(5,1,4);
plot(e1(2000+[1:1000]));
title('Residual Error (music)');
xlabel('Time');
ylabel('Amplitude');
subplot(5,1,5);
plot(rde1(2000+[1:1000]));
title('Recovered Signal (speech)');
xlabel('Time');
ylabel('Amplitude');
```

13. Display other portions of the "d", "e", "rde", "e1" and "rde1" data.

14. Experiment with different audio/speech files.